Coding Assignments #5 (Part One)

August 4, 2025

Directions:

1. You will be penalized for not following each of the directions exactly as written below. If you aren't sure about what something means, ask the professor!
2. Put all your work in a single Haskell file.
3. Submit your work (before the deadline) to Canvas.
4. **Important: You must avoid global helper functions**. Keep the scope of your helper functions limited by using `let`s and `where`s.
5. **Important:** You will be graded on excessive use of helper functions when patterns or guards would make more sense.
6. **Important:** You will lose points **each time** you use the `if` keyword. Use patterns and guards instead!

1. (Call your function **reduce**). Using only techniques from chapters 1-**5a**, write the reduce function. The reduce function has the following type: `Eq v => (v -> v -> v) -> [(k, v)] -> [(k, v)]`. This function takes a function as its first argument and a list of pairs as its second. The function *reduces* pairs that have the same `k` value (e.g., `(k*, v1), (k*, v2)`) by retaining just a single pair that combines the `v` values of the two pairs with the same key (e.g., `(k*, (f v1 v2))`). For example, `reduce (+) [('a', 1), ('a', 3), ('b', 2), ('c', 1), ('b', 1)]` gives the list `[('a', 4), ('b' 3), ('c', 1)]`. **Hint:** You may find it useful to write a function `incorporate f list (k, v)` which takes a single `(k, v)` and incorporates it into `list` by finding the first instance of `(k, v2)` in the list and replacing the pair it belongs to with `(k, (f v v2))`. Make sure that `incorporate f [] (k, v)` gives `[(k, v)]` as an answer.
2. (Store your result in **sentence**). Make sure your program has the line `sentence = "mysterious visitor from future wins lottery again"`.

3. (Store your result in **vowels**) Using only techniques from chapters 1-**5a**, write an expression that gives the number of vowels in sentence. **Hints:** (1) `vowels` should be 17; (2) The operator (,1) makes a pair of its argument and 1: e.g., `(,1) 3` is `(1,3)`; (3) There are only two categories: characters that are ('elem' "aieou") and characters that aren't.

4. (Call your function **closure**). Using only techniques from chapters 1-**5a**, write a function that computes a reachability closure on a *graph*. A graph is a network consisting of *nodes* and *edges*. Nodes are the objects in the networks (e.g., routers, people, street-intersections), and edges are the connections between objects (e.g., wires connecting two routers, friendships between two people, roads connecting two intersections). The `closure` function takes two arguments: (1) a list of edges (pairs of two integers, where integers represent ids of objects in the network); (2) a list of known *reachable objects* from an unspecified source. An object is reachable from another object if, by following edges, a path can be traced from the first object to the second. The function must compute the complete list of reachable objects from the source.

   For example,

```
a = closure [(1, 2), (1, 3), (2, 4), (3, 4)] [1]
b = closure [(1, 2), (1, 3), (2, 4), (3, 4)] [2]
c = closure [(1, 2), (1, 3), (2, 4), (3, 4)] [4]
```

   `a` will be `[1,2,3,4]`; `b` will be `[2,4]`; and `c` will be `[4]`.

   Hint: (1) Put `import Data.List (sort, nub)` at the top of your file. The function `sort` sorts a list and `nub` removes duplicate values from a list. You will find both of these functions essential. (2) The function will need to be recursive, and each recursive step computes a list. The function terminates when the list computed is the same as the input list of reachable objects.